

Insurance AI Lab Guide

Module 2: Context Windows, Chunking Strategies & Chunk Size Experiments

AWS EC2 Ubuntu | Code-Server IDE | OpenAI API | LangChain

Lab	Topic	Stack	Time
Lab 6	Context Window & Truncation Experiment	OpenAI API	~40 min
Lab 7	Chunking Strategies Comparison	LangChain Text Splitters	~45 min
Lab 8	Chunk Size Experiment (256 / 512 / 1024)	LangChain + FAISS	~50 min

■ Prerequisites	■ What You Will Learn
• Labs 1–5 completed (Module 1) • AWS EC2 Ubuntu instance running • Code-Server accessible in browser • Virtual environment activated • OpenAI API key in .env file • langchain, langchain-openai installed	• Why context window limits matter • How truncation degrades LLM answers • 5 chunking strategies & their trade-offs • How chunk size affects retrieval quality • How to select the right splitter for insurance docs • FAISS vector store basics for similarity search

Table of Contents

Background: Context Windows & Chunking Concepts	Theory before you code	p. 3
Lab 6 — Context Window & Truncation Experiment	Load large doc, test truncation, observe degradation	p. 5
Lab 7 — Chunking Strategies Comparison	CharacterSplitter, RecursiveCharacter, Token, Semantic, Markdown	p. 11
Lab 8 — Chunk Size Experiment	Compare 256 / 512 / 1024 token chunks, evaluate retrieval quality	p. 19
Evaluation Framework & Scoring	How to objectively compare strategies and chunk sizes	p. 26
Troubleshooting & Quick Reference	Errors, fixes, key terms	p. 28

Background: Context Windows & Chunking

Core theory before the hands-on labs

What is a Context Window?

Every LLM has a maximum number of tokens it can process in a single request — this is called the **context window**. For GPT-4o-mini the limit is 128K tokens. One token is roughly 0.75 English words. A large insurance policy PDF may contain 50,000–200,000+ tokens, which far exceeds what you can safely send in one request while leaving room for the model's response.

Document Type	Est. Words	Est. Tokens	Fits in 4K ctx?	Fits in 128K ctx?
Single policy clause	~150	~200	■ Yes	■ Yes
Health policy summary	~3,000	~4,000	■■ Barely	■ Yes
Full health policy doc	~20,000	~27,000	■ No	■ Yes
Master policy + riders	~60,000	~80,000	■ No	■■ With care
Complete product manual	~150,000	~200,000	■ No	■ No

What Happens When You Truncate?

When you naively slice a document to fit the context window, critical information that lies beyond the cutoff point is simply discarded. The LLM then answers as if that information does not exist — leading to incomplete, incorrect, or confidently wrong answers. This is called **context truncation degradation** and is one of the most common errors in early RAG implementations.

The Solution: Chunking + Retrieval

Instead of stuffing the whole document into one request, we **split** it into smaller overlapping pieces (chunks), convert each chunk to a vector embedding, store them in a vector database, and at query time **retrieve only the most relevant chunks** to include in the prompt. This makes even a 200-page document queryable.

5 Chunking Strategies at a Glance

CharacterTextSplitter	Splits on a fixed character (e.g. newline). Fast but ignores semantics.	<i>Best for: Simple docs, plain text</i>
RecursiveCharacterTextSplitter	Tries to split on paragraphs → sentences → words in order. Respects natural boundaries.	<i>Best for: Most documents — recommended default</i>
TokenTextSplitter	Splits exactly on token count using tiktoken. Precise for API budgeting.	<i>Best for: When token limits are strict</i>
MarkdownHeaderTextSplitter	Splits on Markdown headings (#, ##, ###). Preserves document structure.	<i>Best for: Docling-extracted Markdown policy docs</i>

SemanticChunker	Uses embedding similarity to split at topic boundaries. Highest quality.	Best for: Production RAG, when latency allows
-----------------	--	---

Understanding Chunk Size vs Overlap

Parameter	Small Value	Large Value	Insurance Doc Recommendation
chunk_size	More chunks, precise retrieval, may split sentences	Fewer chunks, more context per chunk, may lose context	512 tokens (split at sentence boundaries)
chunk_overlap	Risk of losing context at boundaries	Redundant retrieval but preserves context	10-20% of chunk_size (e.g. 50–100 tokens for 512)

Lab 6 — Context Window & Truncation Experiment

Load a large insurance document and observe how truncation degrades answers

■ Lab Objectives

- Generate a large multi-section insurance policy document (>8,000 tokens)
- Count tokens using tiktoken to understand document size
- Query the document using the FULL context
- Query using progressively truncated context (75%, 50%, 25%)
- Record and compare answer quality across truncation levels
- Understand at what truncation point answers start to degrade

Step 1 — Setup

```
cd ~/insurance_ai_lab
mkdir -p lab6 && cd lab6
pip install tiktoken --quiet
touch truncation_experiment.py
code truncation_experiment.py
```

Step 2 — Imports and API Setup

```
# truncation_experiment.py
import os, time, json
from dotenv import load_dotenv
import tiktoken
from openai import OpenAI

load_dotenv(dotenv_path="/root/insurance_ai_lab/.env")
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
enc = tiktoken.encoding_for_model("gpt-4o-mini")

def count_tokens(text: str) -> int:
    return len(enc.encode(text))
```

Step 3 — Generate a Large Insurance Policy Document

We build a realistic ~8,000-token insurance policy programmatically so every participant uses the same document and results are reproducible.

```
def build_large_policy() -> str:
    """Build a realistic ~8,000 token insurance policy document.
    sections = {
    "SECTION 1 - DEFINITIONS":
    Insured Person: Any person named in the Schedule as an Insured Person.
    Sum Insured: The maximum amount payable by the Company during a Policy Year.
    Pre-Existing Disease (PED): Any condition, ailment, injury or disease diagnosed
```

within 48 months prior to first policy issuance. PED includes diabetes mellitus type 1 and 2, hypertension, cardiac conditions, thyroid disorders, asthma, and any other chronic condition requiring ongoing treatment or monitoring.

Network Hospital: A Hospital empanelled with the Company for cashless treatment.

Day Care Treatment: Medical treatment or surgical procedure that requires less than 24 hours hospitalisation due to technological advancements. Over 540 day-care procedures are covered under this policy.,

"SECTION 2 - COVERAGE AND BENEFITS":

2.1 In-Patient Hospitalisation: Expenses for treatment requiring continuous hospitalisation for a minimum period of 24 hours. Covered expenses include: room rent up to single AC room, ICU charges, surgeon and anaesthetist fees, diagnostic tests, medicines, blood, oxygen, OT charges, and implants.

2.2 Pre-Hospitalisation: Medical expenses incurred 60 days prior to admission related to the condition for which hospitalisation occurred.

2.3 Post-Hospitalisation: Medical expenses incurred 90 days after discharge related to the condition for which hospitalisation occurred.

2.4 Organ Donor Expenses: Medical expenses of the organ donor for harvesting of the donated organ covered up to Sum Insured.

2.5 Maternity Benefit: Available after 2 policy years. Normal delivery Rs 50,000. Caesarean delivery Rs 75,000. Newborn baby covered from day 1.,

"SECTION 3 - EXCLUSIONS":

3.1 Pre-Existing Diseases: Excluded for 36 months from policy inception.

3.2 Specific Disease Waiting Period: 24-month waiting period applies to: cataracts, benign prostatic hypertrophy, hernia, fistula in anus, piles, sinusitis, tonsillitis, joint replacement surgery, and varicose veins.

3.3 Cosmetic and Aesthetic Treatments: Any surgery or treatment to change physical appearance unless medically necessary due to an accident.

3.4 Dental Treatment: Dental procedures unless requiring hospitalisation.

3.5 Infertility Treatments: IVF, IUI, ICSI and all assisted reproduction.

3.6 Mental Illness: Psychiatric and psychosomatic disorders excluded under base plan. Available as an optional add-on rider at additional premium.,

"SECTION 4 - PREMIUM AND RENEWAL":

4.1 Premium Calculation: Premium is calculated based on Sum Insured, age of eldest member, number of members, zone of residence, and medical history. Zone A covers Mumbai, Delhi, Bangalore, Chennai, Hyderabad, Kolkata. Zone B covers all other cities and towns across India.

4.2 No Claim Bonus (NCB): 5% increase in Sum Insured for each claim-free year, up to a maximum of 50% of original Sum Insured, at no extra premium.

4.3 Cumulative Bonus Protector: NCB is protected even if one claim is filed, subject to selection of this optional rider at policy inception.

4.4 Grace Period: 30 days from policy expiry date for renewal.,

"SECTION 5 - CLAIMS PROCEDURE":

5.1 Cashless Claims: Pre-authorisation required from TPA at least 48 hours before planned admission. For emergencies, inform within 24 hours of admission.

5.2 Reimbursement Claims: Submit all original documents within 30 days of discharge. Required documents: discharge summary, all bills and receipts,

```

diagnostic reports, prescription, and completed claim form.
5.3 Claim Settlement Timeline: Company shall settle or reject within 30 days
of receiving all documents. If surveyor appointed, within 45 days.
5.4 Grievance Redressal: Lodge complaint via toll-free 1800-XXX-XXXX or
grievance@insuresafe.com. Escalate to IRDAI Bima Bharosa portal if unresolved.,
"SECTION 6 - SPECIAL CONDITIONS AND ADD-ONS":
6.1 Restoration Benefit: Full Sum Insured automatically reinstated once per year
for unrelated illnesses after complete exhaustion. Does not apply to same illness.
6.2 Air Ambulance: Covered up to Rs 2,50,000 per policy year for life-threatening
emergencies requiring inter-city transfer.
6.3 Second Opinion: One free second medical opinion per policy year for
critical illnesses including cancer, cardiac, neurological conditions.
6.4 Preventive Health Check: Annual health check-up at empanelled centres
covered up to Rs 2,000 per adult member per year.,
}

doc_parts = ["InsureSafe HealthGuard Gold Policy – Policy Wording Document\n"]
for section, content in sections.items():
    sep = "="*70
    doc_parts.append(f"\n{sep}\n{section}\n{sep}\n{content}\n")
# Repeat content to simulate a real multi-page document
full_doc = "\n".join(doc_parts)
while count_tokens(full_doc) < 7500:
    full_doc += "\n[This page intentionally contains expanded policy schedules, "
    full_doc += "rate tables, and annexures as part of the complete policy wording.]"
    full_doc += " " * 200
return full_doc

```

Step 4 — Token Counter and Truncation Helper

```

def truncate_to_tokens(text: str, max_tokens: int) -> tuple[str, int]:
    """Truncate text to max_tokens. Returns (truncated_text, actual_token_count).
    tokens = enc.encode(text)
    truncated_tokens = tokens[:max_tokens]
    truncated_text = enc.decode(truncated_tokens)
    return truncated_text, len(truncated_tokens)

def query_with_context(context: str, question: str, label: str) -> dict:
    """Send question + context to OpenAI. Returns response and metadata.
    token_count = count_tokens(context)
    print(f"\n[{label}] Sending {token_count:,} tokens...")
    start = time.time()
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system",
            "content": "Answer the question based ONLY on the provided document. "
            "If the answer is not in the document, say 'NOT FOUND IN CONTEXT'."},

```

```

{"role": "user",
 "content": f"DOCUMENT:\n{context}\n\nQUESTION: {question}",
},
temperature=0,
max_tokens=300
)
elapsed = time.time() - start
answer = response.choices[0].message.content
return {
    "label": label,
    "tokens_sent": token_count,
    "answer": answer,
    "latency_s": round(elapsed, 2),
    "found": "NOT FOUND IN CONTEXT" not in answer
}

```

Step 5 — Run the Truncation Experiment

We ask a question whose answer lives in Section 5 (Claims Procedure) — which will disappear from context as we truncate. This makes degradation visible and measurable.

```

def run_experiment():
    full_doc = build_large_policy()
    total_tokens = count_tokens(full_doc)
    print(f"Document size: {total_tokens:,} tokens")

    # Question whose answer is in Section 5 (claims, late in the document)
    question = (
        "What is the timeline for claim settlement after all documents are submitted? "
        "Also, how many days after discharge must reimbursement documents be submitted?"
    )

    # Test levels: 100%, 75%, 50%, 25% of total tokens
    levels = {
        "100% (Full)": total_tokens,
        "75%": int(total_tokens * 0.75),
        "50%": int(total_tokens * 0.50),
        "25%": int(total_tokens * 0.25),
    }

    results = []
    for label, max_tok in levels.items():
        ctx, actual = truncate_to_tokens(full_doc, max_tok)
        result = query_with_context(ctx, question, label)
        results.append(result)
        time.sleep(1) # Rate limit safety

    # Print comparison table
    print("\n" + "="*70)
    print("TRUNCATION EXPERIMENT RESULTS")

```



```
print("="*70)
print(f"Question: {question[:80]}...")
print("Level Tokens Found? Latency")
print("-"*50)
for r in results:
    found_str = "■ YES" if r["found"] else "■ NO"
    print(f"{r['label']:<12} {r['tokens_sent']:>8,} {found_str:>8} {r['latency_s']:>9.1f}s")

# Save detailed results
with open("truncation_results.json", "w") as f:
    json.dump(results, f, indent=2)
print("\nDetailed results saved to truncation_results.json")

if __name__ == "__main__":
    run_experiment()
```

Step 6 — Run and Record Results

```
python truncation_experiment.py
```

Expected Output and Observation Table

Context Level	Tokens Sent	Claims Info Found?	Your Observation
100% (Full)	~7,500	■ Yes — correct answer	_____
75%	~5,625	■ Yes — likely still present	_____
50%	~3,750	■■ Partial or missing	_____
25%	~1,875	■ NOT FOUND IN CONTEXT	_____

■ Key Insight

The claims procedure info lives in Section 5, which is roughly 70-80% into the document. When you truncate to 50%, Section 5 is cut off entirely — the model has no choice but to say 'NOT FOUND IN CONTEXT'. This is why naive truncation fails for long documents.

■ Challenge Task

Add a second question whose answer lives in Section 2 (Coverage — early in the doc). Observe that this question is answered correctly even at 25% context. This demonstrates the "lost-in-the-middle" problem: content at the start and end of context is retrieved better than content in the middle.

Lab 7 — Chunking Strategies Comparison

Implement 5 splitters and compare retrieval quality on insurance documents

■ Lab Objectives

- Implement CharacterTextSplitter with custom separators
- Implement RecursiveCharacterTextSplitter (recommended default)
- Implement TokenTextSplitter for precise token-based splitting
- Implement MarkdownHeaderTextSplitter for structured policy docs
- Build a FAISS vector store and run similarity search on each strategy
- Compare chunk count, average chunk size, and retrieval relevance

Step 1 — Setup

```
cd ~/insurance_ai_lab
mkdir -p lab7 && cd lab7
pip install faiss-cpu langchain-openai tiktoken --quiet
touch chunking_strategies.py
code chunking_strategies.py
```

Step 2 — Imports

```
# chunking_strategies.py
import os, json
from dotenv import load_dotenv
from langchain.text_splitter import (
    CharacterTextSplitter,
    RecursiveCharacterTextSplitter,
    TokenTextSplitter,
    MarkdownHeaderTextSplitter,
)
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_core.documents import Document
import tiktoken

load_dotenv(dotenv_path="/root/insurance_ai_lab/.env")
enc = tiktoken.encoding_for_model("gpt-4o-mini")
embeddings = OpenAIEmbeddings(openai_api_key=os.getenv("OPENAI_API_KEY"))
```

Step 3 — The Test Document (Markdown format)

We use a Markdown-formatted policy document so the MarkdownHeaderSplitter can also be tested. This is realistic because Docling outputs Markdown.

```
POLICY_MARKDOWN =
# InsureSafe HealthGuard Gold — Policy Wording
```

```

## Section 1: Definitions
**Insured Person** means any person named in the Schedule.
**Sum Insured** is the maximum payable amount during a Policy Year.
**Pre-Existing Disease** means any condition diagnosed within 48 months prior
to first policy issuance, including diabetes, hypertension, and cardiac conditions.
**Network Hospital** means a hospital with a cashless agreement with the insurer.

## Section 2: Coverage and Benefits
### 2.1 In-Patient Hospitalisation
Covered for minimum 24-hour admission. Room rent: single AC room.
ICU, surgeon, anaesthetist, medicines, diagnostics all covered up to Sum Insured.
### 2.2 Pre and Post Hospitalisation
Pre-hospitalisation: 60 days. Post-hospitalisation: 90 days.
### 2.3 Maternity Benefit
Available after 2 years. Normal: Rs 50,000. Caesarean: Rs 75,000.

## Section 3: Exclusions
### 3.1 Pre-Existing Diseases
36-month waiting period from policy inception date.
### 3.2 Specific Disease Waiting Period
24-month wait for: cataracts, hernia, piles, sinusitis, joint replacement.
### 3.3 Cosmetic Treatments
Not covered unless due to an accident requiring medical necessity.

## Section 4: Premium and Renewal
No Claim Bonus: 5% increase per claim-free year, up to 50% of Sum Insured.
Grace period for renewal: 30 days after policy expiry.
Premium is zonal: Zone A (metros) and Zone B (all other cities).

## Section 5: Claims Procedure
### 5.1 Cashless Claims
Pre-authorisation 48 hours before planned admission. Emergency: 24 hours after.
### 5.2 Reimbursement Claims
Submit documents within 30 days of discharge. Required: discharge summary,
all bills, diagnostic reports, prescription, completed claim form.
### 5.3 Settlement Timeline
Settle or reject within 30 days of complete documents. 45 days if surveyor needed.

```

Step 4 — Strategy 1: CharacterTextSplitter

■ Strategy 1

Splits on a specific character separator (default: newline). Simple but ignores semantic boundaries. Risk of splitting mid-sentence if separator not chosen carefully.

```

def strategy_character(text: str) -> list:
    splitter = CharacterTextSplitter(
        separator="\n\n", # Split on blank lines (paragraphs)
        chunk_size=500,
        chunk_overlap=50,
        length_function=len,
        is_separator_regex=False
    )

```

```

)
chunks = splitter.create_documents([text])
print(f"CharacterSplitter: {len(chunks)} chunks")
return chunks

```

Step 5 — Strategy 2: RecursiveCharacterTextSplitter

■ Strategy 2

Tries to split on [paragraph, sentence, word] in order — always producing the most semantically coherent chunks possible. This is the recommended default for most documents.

```

def strategy_recursive(text: str) -> list:
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=500,
        chunk_overlap=50,
        length_function=len,
        separators=["\n\n", "\n", ". ", " ", ""],
    )
    chunks = splitter.create_documents([text])
    print(f"RecursiveSplitter: {len(chunks)} chunks")
    return chunks

```

Step 6 — Strategy 3: TokenTextSplitter

■ Strategy 3

Splits exactly on token count, not character count. Essential when you need precise control over OpenAI API costs or when filling a context window exactly.

```

def strategy_token(text: str) -> list:
    splitter = TokenTextSplitter(
        chunk_size=128, # 128 tokens per chunk (~96 words)
        chunk_overlap=16
    )
    chunks = splitter.create_documents([text])
    print(f"TokenSplitter: {len(chunks)} chunks")
    for i, c in enumerate(chunks[:2]):
        tok = len(enc.encode(c.page_content))
        print(f"Chunk {i}: {tok} tokens")
    return chunks

```

Step 7 — Strategy 4: MarkdownHeaderTextSplitter

■ Strategy 4

Splits Markdown documents at heading boundaries (#, ##, ###). Each chunk carries its section header as metadata — ideal for Docling-extracted insurance policy documents.

```

def strategy_markdown(markdown_text: str) -> list:
    headers_to_split = [
        ("#", "document_title"),
        ("##", "section"),

```

```
(####, "subsection"),
]
splitter = MarkdownHeaderTextSplitter(
    headers_to_split_on=headers_to_split,
    strip_headers=False # Keep headers in chunk text
)
chunks = splitter.split_text(markdown_text)
print(f"MarkdownSplitter: {len(chunks)} chunks")
# Show metadata for first 3 chunks
for i, c in enumerate(chunks[:3]):
    print(f" Chunk {i} metadata: {c.metadata}")
return chunks
```

Step 8 — Build FAISS Vector Stores and Run Retrieval

Now we embed each set of chunks and run the same test query to compare which strategy retrieves the most relevant chunks for an insurance question.

```
def evaluate_strategy(strategy_name: str, chunks: list, query: str) -> dict:
    """Build FAISS index, retrieve top-3 chunks, return metrics.
    print(f"\n--- Evaluating: {strategy_name} ---")
    # Convert to LangChain Document objects if not already
    docs = [Document(page_content=c if isinstance(c, str) else c.page_content,
        metadata=c.metadata if hasattr(c, "metadata") else {})]
    for c in chunks]
    # Build FAISS vector store from chunks
    vectorstore = FAISS.from_documents(docs, embeddings)
    # Retrieve top 3 most relevant chunks
    retrieved = vectorstore.similarity_search(query, k=3)
    # Calculate token counts
    chunk_tokens = [len(enc.encode(d.page_content)) for d in docs]
    avg_tokens = sum(chunk_tokens) / len(chunk_tokens)
    print(f" Total chunks: {len(docs)}")
    print(f" Avg tokens/chunk: {avg_tokens:.0f}")
    print(f" Top retrieved chunk preview: {retrieved[0].page_content[:120]}...")
    return {
        "strategy": strategy_name,
        "total_chunks": len(docs),
        "avg_tokens": round(avg_tokens, 1),
        "top_chunk_preview": retrieved[0].page_content[:200],
        "top_chunk_metadata": retrieved[0].metadata,
    }
```

Step 9 — Main Runner and Comparison Report

```
def main():
```

```

query = "What is the claims settlement timeline and document submission deadline?"
print(f"Test Query: {query}\n")

strategies = [
    ("CharacterSplitter", strategy_character(POLICY_MARKDOWN)),
    ("RecursiveSplitter", strategy_recursive(POLICY_MARKDOWN)),
    ("TokenSplitter", strategy_token(POLICY_MARKDOWN)),
    ("MarkdownSplitter", strategy_markdown(POLICY_MARKDOWN)),
]

all_results = []
for name, chunks in strategies:
    result = evaluate_strategy(name, chunks, query)
    all_results.append(result)

# Print comparison table
print("\n" + "="*60)
print("STRATEGY COMPARISON SUMMARY")
print("="*60)
print("Strategy Chunks Avg Tokens")
print("-"*45)
for r in all_results:
    print(f"{r['strategy']:<22} {r['total_chunks']:>8} {r['avg_tokens']:>12.0f}")

with open("strategy_results.json", "w") as f:
    json.dump(all_results, f, indent=2)

if __name__ == "__main__":
    main()

```

Step 10 — Run

```
python chunking_strategies.py
```

Observation Worksheet

Strategy	Chunks	Avg Tokens	Top Chunk Relevant?	Your Notes
CharacterSplitter	_____	_____	■ Yes ■ No	_____
RecursiveSplitter	_____	_____	■ Yes ■ No	_____
TokenSplitter	_____	_____	■ Yes ■ No	_____
MarkdownSplitter	_____	_____	■ Yes ■ No	_____
■ Expected Finding	MarkdownHeaderSplitter produces the cleanest, most semantically coherent chunks because it respects the policy's section structure. RecursiveSplitter is a close second. CharacterSplitter may split sentences mid-way. TokenSplitter gives the most uniform sizes.			
■ Challenge Task				

Open `strategy_results.json` and inspect the `top_chunk_preview` field for each strategy. Does the `CharacterSplitter` ever split in the middle of a clause? Does the `MarkdownSplitter` correctly isolate the Claims Procedure section? Add a 5th strategy: `RecursiveCharacterTextSplitter` with custom separators `####, ##, #, \n\n, \n` tuned specifically for the policy Markdown structure.

Lab 8 — Chunk Size Experiment (256 / 512 / 1024 Tokens)

Systematically measure how chunk size affects retrieval quality and answer accuracy

■ Lab Objectives

- Split the same document at chunk sizes 256, 512, and 1024 tokens
- Build a FAISS vector store for each configuration
- Run a structured set of 4 test queries across all 3 configurations
- Measure: chunk count, average tokens per chunk, retrieved context size
- Feed retrieved chunks to OpenAI and score answer completeness
- Produce a final comparison matrix and draw conclusions

Why Chunk Size Matters

Chunk Size	Advantages	Disadvantages	Ideal For
256 tokens (~190 words)	• Very targeted retrieval • Low noise per chunk • Fast embedding	• Splits long clauses • Needs more overlap • More API calls	Fact extraction, specific clause lookup
512 tokens (~380 words)	• Balanced precision • Preserves clauses • Recommended baseline	• May include slight irrelevance	General Q&A, insurance policy RAG (default)
1024 tokens (~770 words)	• More context per chunk • Fewer chunks total • Full paragraphs	• Dilutes relevance • Higher token cost per call	Summarisation, broad coverage queries

Step 1 — Setup

```
cd ~/insurance_ai_lab
mkdir -p lab8 && cd lab8
touch chunk_size_experiment.py
code chunk_size_experiment.py
```

Step 2 — Imports and Shared Setup

```
# chunk_size_experiment.py
import os, json, time
from dotenv import load_dotenv
import tiktoken
from openai import OpenAI
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_core.documents import Document
```



```
load_dotenv(dotenv_path="/root/insurance_ai_lab/.env")
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
embeddings = OpenAIEmbeddings(openai_api_key=os.getenv("OPENAI_API_KEY"))
enc = tiktoken.encoding_for_model("gpt-4o-mini")
def tok(t): return len(enc.encode(t))
```

Step 3 — Reuse the Policy Document from Lab 7

Copy the POLICY_MARKDOWN constant from lab7/chunking_strategies.py into this file or import it. For brevity we reference it as POLICY_MARKDOWN below.

```
# Copy POLICY_MARKDOWN from lab7 or paste it here directly
# You can also import from the lab7 module:
import sys
sys.path.insert(0, "../lab7")
from chunking_strategies import POLICY_MARKDOWN
```

Step 4 — Build Indexed Vector Stores for Each Chunk Size

```
CHUNK_SIZES = [256, 512, 1024]
OVERLAP_RATIO = 0.15 # 15% overlap (e.g. 77 for 512)
def build_index(text: str, chunk_size: int) -> tuple:
    """Split text, embed chunks, return (vectorstore, chunk_stats).
    overlap = int(chunk_size * OVERLAP_RATIO)
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size * 4, # *4 because splitter uses chars not tokens
        chunk_overlap=overlap * 4,
        length_function=len,
    )
    raw_chunks = splitter.create_documents([text])
    # Post-filter: keep only chunks within target token range
    docs = [Document(page_content=c.page_content)
             for c in raw_chunks
             if tok(c.page_content) > 10]
    token_counts = [tok(d.page_content) for d in docs]
    stats = {
        "chunk_size_target": chunk_size,
        "total_chunks": len(docs),
        "avg_tokens": round(sum(token_counts)/len(token_counts), 1),
        "min_tokens": min(token_counts),
        "max_tokens": max(token_counts),
    }
    print(f"[{chunk_size} token target] {stats['total_chunks']} chunks, avg {stats['avg_tokens']:.0f} tokens")
    vs = FAISS.from_documents(docs, embeddings)
    return vs, stats
```

Step 5 — Retrieval + LLM Answer Function

```
def retrieve_and_answer(vectorstore, query: str, top_k: int = 3) -> dict:
    """Retrieve relevant chunks and generate an answer using OpenAI.
    retrieved = vectorstore.similarity_search(query, k=top_k)
    context = "\n\n---\n\n".join([d.page_content for d in retrieved])
    context_tokens = tok(context)

    start = time.time()
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system",
             "content": "Answer based ONLY on the provided context. "
             "Be specific. Say NOT FOUND if answer is absent."},
            {"role": "user",
             "content": f"CONTEXT:\n{context}\n\nQUESTION: {query}"},
        ],
        temperature=0, max_tokens=200
    )
    latency = round(time.time() - start, 2)
    answer = response.choices[0].message.content
    return {
        "answer": answer,
        "context_tokens": context_tokens,
        "latency_s": latency,
        "found": "NOT FOUND" not in answer.upper(),
    }
```

Step 6 — Define 4 Test Queries

Each query is designed to test a different aspect — some answers are in early sections, some in late sections, and one is a cross-section synthesis question.

```
TEST_QUERIES = [
    {
        "id": "Q1",
        "question": "What is the waiting period for pre-existing diseases?",
        "location": "Section 3 (Exclusions)",
        "expected": "36 months"
    },
    {
        "id": "Q2",
        "question": "What is the maternity benefit amount for normal delivery?",
        "location": "Section 2.3",
        "expected": "Rs 50,000"
    },
]
```

```
{
  "id": "Q3",
  "question": "How many days after discharge must I submit reimbursement documents?",
  "location": "Section 5.2",
  "expected": "30 days"
},
{
  "id": "Q4",
  "question": "What no-claim bonus does the policy offer and what is the maximum NCB?",
  "location": "Section 4",
  "expected": "5% per year, max 50%"
},
]
```

Step 7 — Main Experiment Loop

```
def main():
    all_results = []

    # Phase 1: Build all three indexes
    print("=== BUILDING VECTOR INDEXES ===")
    indexes = {}
    for size in CHUNK_SIZES:
        vs, stats = build_index(POLICY_MARKDOWN, size)
        indexes[size] = {"vs": vs, "stats": stats}

    # Phase 2: Run all queries against all indexes
    print("\n=== RUNNING QUERIES ===")
    for q_info in TEST_QUERIES:
        q_result = {"query_id": q_info["id"], "question": q_info["question"],
                    "expected": q_info["expected"], "results_by_size": {}}
        for size in CHUNK_SIZES:
            vs = indexes[size]["vs"]
            result = retrieve_and_answer(vs, q_info["question"])
            # Auto-score: check if expected value appears in answer
            score = 1 if q_info["expected"].lower()
                in result["answer"].lower() else 0
            result["score"] = score
            q_result["results_by_size"][size] = result
        print(f" {q_info['id']} @ {size} tokens: score={score}, latency={result['latency_s']}s")
        time.sleep(0.5)
    all_results.append(q_result)

    # Phase 3: Print final matrix
    _print_matrix(all_results, indexes)

    with open("chunk_experiment_results.json", "w") as f:
        json.dump({"results": all_results}, f, indent=2, default=str)
```

Step 8 — Print the Comparison Matrix

```
def _print_matrix(results, indexes):
    print("\n" + "="*72)
    print("CHUNK SIZE EXPERIMENT — FINAL MATRIX")
    print("="*72)
    # Index stats header
    print(" 256 tokens 512 tokens 1024 tokens")
    print(f"Total Chunks ", end="")
    for s in CHUNK_SIZES:
        print(f"{indexes[s]['stats']['total_chunks']:>15}", end="")
    print()
    print(f"Avg Tokens/Chunk ", end="")
    for s in CHUNK_SIZES:
        print(f"{indexes[s]['stats']['avg_tokens']:>15.0f}", end="")
    print()
    print("-"*72)
    # Per-query scores
    for r in results:
        q_label = f"{r['query_id']} ({r['expected']})"
        print(f"{q_label:20}", end="")
        for s in CHUNK_SIZES:
            sc = r["results_by_size"][s]["score"]
            icon = "■" if sc == 1 else "□"
            print(f"{icon:>15}", end="")
        print()
    # Total scores per chunk size
    print("-"*72)
    print(f"TOTAL SCORE /4 ", end="")
    for s in CHUNK_SIZES:
        total = sum(r["results_by_size"][s]["score"] for r in results)
        print(f"{total}/4:>15", end="")
    print("\n")
    if __name__ == "__main__":
        main()
```

Step 9 — Run the Experiment

```
python chunk_size_experiment.py
```

Expected Results Matrix

Metric	256 Tokens	512 Tokens	1024 Tokens
Total Chunks	~18–22	~10–12	~5–7
Avg Tokens/Chunk	~180–220	~350–420	~600–800

Q1 — PED Waiting Period	■	■	■
Q2 — Maternity Amount	■	■	■
Q3 — Reimbursement Deadline	■ / ■■	■	■
Q4 — NCB Details	■■■ / ■	■	■
Total Score /4	3–4	4/4	3–4
Context Tokens Sent	~800–1200	~1200–1800	~2500–3500
Best For	Precise clause lookup	■ General RAG (recommended)	Broad summaries

■ Key Conclusion

512 tokens with 15% overlap is the sweet spot for insurance policy RAG. 256 tokens may split multi-line clauses (e.g. a premium calculation table), while 1024 tokens dilutes retrieval precision by bundling unrelated clauses together. Always test 3–4 chunk sizes on your actual document before deploying to production.

■ Challenge Task

Add chunk_size=128 and chunk_size=2048 to the experiment. Observe the extremes: at 128 tokens, clauses get cut mid-sentence; at 2048 tokens, retrieval brings back almost the entire document for every query. Also try changing top_k from 3 to 5 for the 256-token case and observe if scores improve.

Evaluation Framework & Scoring Guide

How to objectively assess chunking strategy and chunk size quality

Why Systematic Evaluation Matters

In production insurance AI systems, retrieval quality directly impacts regulatory compliance and customer trust. A missed exclusion clause or incorrect waiting period can have serious consequences. You should always evaluate your chunking pipeline rigorously before deploying. This section gives you a repeatable evaluation framework.

Evaluation Dimensions

Recall@K	Does the answer contain the expected fact?	How measured: Auto-scored in Lab 8 by checking if <code>expected_value</code> in answer
Precision	Is the retrieved context focused on the query topic?	How measured: Manually inspect: how much irrelevant text is in the context?
Context Efficiency	How many tokens did we send to produce a correct answer?	How measured: Lower is better. Tracked as <code>context_tokens</code> in Lab 8
Boundary Integrity	Do chunks avoid splitting mid-clause or mid-table?	How measured: Manually inspect chunk content printed in Labs 7 and 8
Latency	How fast does the full pipeline respond?	How measured: Tracked as <code>latency_s</code> in Lab 8. Larger chunks = fewer API calls = faster
Cost per Query	How many tokens per query × token price?	How measured: <code>context_tokens * (price per 1K tokens)</code> . Smaller chunks win

Decision Guide: Which Splitter to Use?

Situation	Recommended Splitter	Chunk Size
Plain-text insurance document from OCR	RecursiveCharacterTextSplitter	512 tokens
Docling-extracted Markdown policy	MarkdownHeaderTextSplitter + RecursiveCharacter	512 tokens
Precise clause retrieval (e.g. exact exclusion)	TokenTextSplitter	256 tokens
Broad summarisation of entire policy	RecursiveCharacterTextSplitter	1024 tokens
Production RAG with quality budget	SemanticChunker (langchain-experimental)	~400–600 tokens adaptive
Quick prototyping / dev mode	RecursiveCharacterTextSplitter (default)	512 tokens

Troubleshooting & Quick Reference

Common errors, fixes, and commands for Labs 6–8

■ ImportError: No module named 'tiktoken'

Cause: tiktoken was not installed in the virtual environment.

Fix: `pip install tiktoken`

■ ValueError: Could not automatically map ... to a tokeniser

Cause: Incorrect model name passed to `tiktoken.encoding_for_model()`.

Fix: Use exact name: `tiktoken.encoding_for_model('gpt-4o-mini')`

■ FAISS: No module named 'faiss'

Cause: `faiss-cpu` not installed.

Fix: `pip install faiss-cpu`

■ openai.RateLimitError during experiment

Cause: Too many API calls too quickly.

Fix: Add `time.sleep(1)` between calls. Use `gpt-4o-mini` to reduce rate limit pressure.

■ MarkdownHeaderTextSplitter returns 0 chunks

Cause: Document is plain text, not Markdown — no `#` headers to split on.

Fix: Check `POLICY_MARKDOWN` has `##` headings. Use `RecursiveSplitter` for plain text.

■ Chunks all have 0 tokens after splitting

Cause: `chunk_size` set too small — smaller than any word in the document.

Fix: Use `chunk_size >= 50` tokens. For character splitters, multiply by 4 (chars per token).

■ FAISS.from_documents hangs

Cause: Each chunk requires an embedding API call. Many small chunks = many calls.

Fix: Reduce chunk count by using larger `chunk_size` for dev. Use `.from_texts()` for faster testing.

Quick Command Reference — Labs 6–8

Action	Command
Run Lab 6 (truncation)	<code>cd ~/insurance_ai_lab/lab6 && python truncation_experiment.py</code>
Run Lab 7 (strategies)	<code>cd ~/insurance_ai_lab/lab7 && python chunking_strategies.py</code>
Run Lab 8 (chunk sizes)	<code>cd ~/insurance_ai_lab/lab8 && python chunk_size_experiment.py</code>

View Lab 6 results	<code>cat ~/insurance_ai_lab/lab6/truncation_results.json python3 -m json.tool</code>
View Lab 7 results	<code>cat ~/insurance_ai_lab/lab7/strategy_results.json python3 -m json.tool</code>
Count tokens in a file	<code>python3 -c "import tiktoken; e=tiktoken.encoding_for_model('gpt-4o-mini'); pri</code>
Check installed packages	<code>pip list grep -E "langchain faiss tiktoken openai"</code>
Activate virtualenv	<code>source ~/insurance_ai_lab/venv/bin/activate</code>

Key Terms Glossary

Term	Definition
Context Window	Maximum tokens an LLM can process in a single request
Truncation	Slicing text to fit the context window, discarding excess content
Chunk	A fragment of a document produced by a text splitter
chunk_size	Target size of each chunk (in characters or tokens)
chunk_overlap	Number of chars/tokens shared between consecutive chunks
Embedding	A numerical vector representing the semantic meaning of a text chunk
FAISS	Facebook AI Similarity Search — in-memory vector database for similarity lookup
Similarity Search	Finding the chunks whose embeddings are closest to the query embedding
RAG	Retrieval-Augmented Generation — retrieve relevant chunks, then generate an answer
Recall@K	Whether the correct answer is found in the top-K retrieved chunks
NCB	No Claim Bonus — premium discount for claim-free policy years
PED	Pre-Existing Disease — condition diagnosed before policy inception

■ **Module 2 Complete!** You now understand context window limits, how truncation degrades LLM answers, and how to systematically compare chunking strategies and chunk sizes. These skills are foundational for building production-grade RAG pipelines on insurance and enterprise documents.